Meeting 3 SP23

Point Cloud Acceleration

@crawfish

Outline

Background

PointAcc

Evaluation

Section 1

Background

Machine Learning

Like terminator but significantly less cool

- *"the use and development of computer systems that are able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyze and draw inferences from patterns in data"*

Machine Learning

Like terminator but significantly less cool

- *"the use and development of computer systems that are able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyze and draw inferences from patterns in data"*
- Hot topic for research in all fields, especially computer architecture

Machine Learning

Like terminator but significantly less cool

- *"the use and development of computer systems that are able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyze and draw inferences from patterns in data"*
- Hot topic for research in all fields, especially computer architecture
  - GPU hardware development

Machine Learning

Like terminator but significantly less cool

- *"the use and development of computer systems that are able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyze and draw inferences from patterns in data"*
- Hot topic for research in all fields, especially computer architecture
  - GPU hardware development
  - Accelerators for machine learning (TPUs, special purpose)

Machine Learning

Like terminator but significantly less cool

- *"the use and development of computer systems that are able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyze and draw inferences from patterns in data"*
- Hot topic for research in all fields, especially computer architecture
  - GPU hardware development
  - Accelerators for machine learning (TPUs, special purpose)
- Incredibly applicable → relevant to just about anything

Point Clouds

- A **point cloud** is collection of points representing a physical object or 3d scene

Point Clouds

- A **point cloud** is collection of points representing a physical object or 3d scene
- Typically generated by sensors (LiDAR, etc.)

Point Clouds

- A **point cloud** is collection of points representing a physical object or 3d scene
- Typically generated by sensors (LiDAR, etc.)
- Used as inputs to real-time ML systems

Point Clouds

- A **point cloud** is collection of points representing a physical object or 3d scene
- Typically generated by sensors (LiDAR, etc.)
- Used as inputs to real-time ML systems
  - Autonomous vehicles

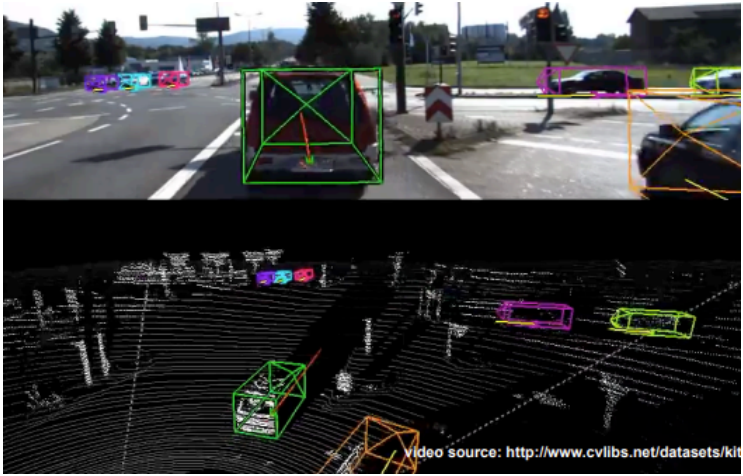
Point Clouds

- A **point cloud** is collection of points representing a physical object or 3d scene
- Typically generated by sensors (LiDAR, etc.)
- Used as inputs to real-time ML systems
  - Autonomous vehicles
  - Image processing

Point Clouds

- A **point cloud** is collection of points representing a physical object or 3d scene
- Typically generated by sensors (LiDAR, etc.)
- Used as inputs to real-time ML systems
  - Autonomous vehicles
  - Image processing
  - AR/VR

AV Driving



Sparsity

- Point clouds have sparse features

Sparsity

- Point clouds have sparse features
- Different from sparsity created by activation functions (ReLU)

Sparsity

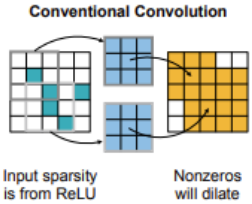
- Point clouds have sparse features
- Different from sparsity created by activation functions (ReLU)
  - ReLU sparsity is manufactured before computation $\rightarrow$ nonzero values will be dilated after computation

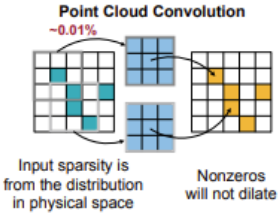
Sparsity

- Point clouds have sparse features
- Different from sparsity created by activation functions (ReLU)
  - ReLU sparsity is manufactured before computation $\rightarrow$ nonzero values will be dilated after computation
  - Point clouds exhibit 'true' sparsity $\rightarrow$ should be no dilation in result

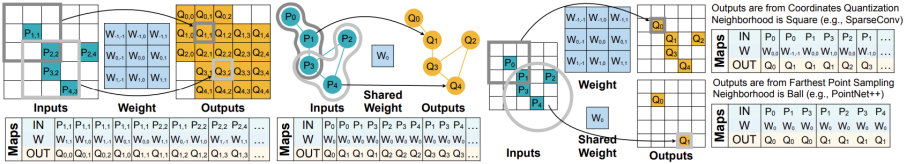
Sparsity

- Point clouds have sparse features
- Different from sparsity created by activation functions (ReLU)
  - ReLU sparsity is manufactured before computation $\rightarrow$ nonzero values will be dilated after computation
  - Point clouds exhibit 'true' sparsity $\rightarrow$ should be no dilation in result





Point Cloud Convolution



(a) Conventional Convolution
(inputs **quickly become denser**)

(b) Graph Convolution
(maps are **given** and **fixed**)

(c) Point Cloud Convolution
(inputs **keep** sparse; maps are **explicitly calculated** during inference)

Point Cloud Convolution

- Conducted on neighborhood of output point as defined by map

Point Cloud Convolution

- Conducted on neighborhood of output point as defined by map
  - Must explicitly calculate this **every iteration** for point clouds, can change during down-sampling

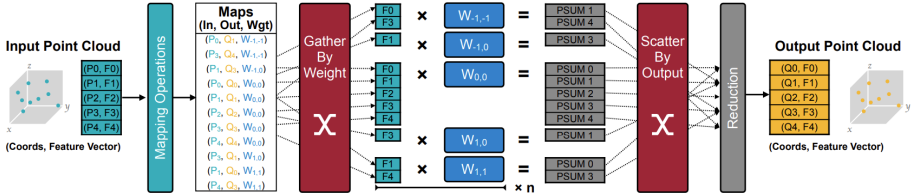
Point Cloud Convolution

- Conducted on neighborhood of output point as defined by map
  - Must explicitly calculate this **every iteration** for point clouds, can change during down-sampling
- Data is irregular, unstructured, difficult to work with

Point Cloud Convolution

- Conducted on neighborhood of output point as defined by map
  - Must explicitly calculate this **every iteration** for point clouds, can change during down-sampling
- Data is irregular, unstructured, difficult to work with
- Much research has been done on improving the point cloud convolution, open problem

Point Cloud Convolution



Challenges

1. Sparsity and irregularity of data

Challenges

1. Sparsity and irregularity of data
2. Mapping operation bottleneck (over 50% total runtime)

Challenges

1. Sparsity and irregularity of data
2. Mapping operation bottleneck (over 50% total runtime)
3. Large memory footprint

Section 2

PointAcc

PointAcc

- Hardware accelerator for point cloud convolutions

PointAcc

- Hardware accelerator for point cloud convolutions
- Why is this needed? What about GPU, TPU?

PointAcc

- Hardware accelerator for point cloud convolutions
- Why is this needed? What about GPU, TPU?
  - Point cloud networks cannot be run efficiently on current GPU designs

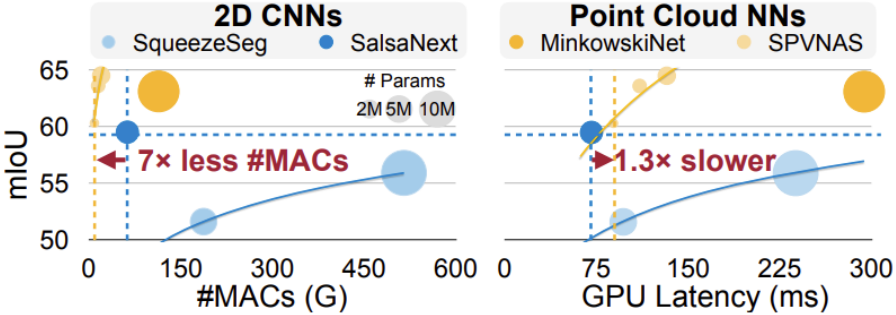
PointAcc

- Hardware accelerator for point cloud convolutions
- Why is this needed? What about GPU, TPU?
  - Point cloud networks cannot be run efficiently on current GPU designs
  - Latency of point clouds is higher on GPU than 2D CNNs $\rightarrow$ but 7x fewer MACs required to evaluate point cloud networks

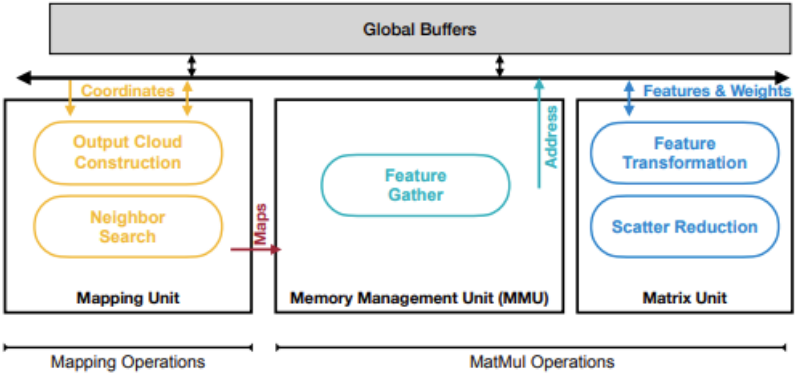
PointAcc

- Hardware accelerator for point cloud convolutions
- Why is this needed? What about GPU, TPU?
  - Point cloud networks cannot be run efficiently on current GPU designs
  - Latency of point clouds is higher on GPU than 2D CNNs $\rightarrow$ but 7x fewer MACs required to evaluate point cloud networks
  - Implies there must be inefficiency somewhere

PointAcc


2D CNNs
SqueezeSeg
SalsaNext
Params
2M 5M 10M
7× less #MACs
mIoU
65
60
55
50
0
150
300
450
600
#MACs (G)
Point Cloud NNs
MinkowskiNet
SPVNAS
1.3× slower
0
75
150
225
300
GPU Latency (ms)

PointAcc


Global Buffers
Coordinates
Features & Weights
Output Cloud Construction
Neighbor Search
Maps
Feature Gather
Address
Feature Transformation
Scatter Reduction
Mapping Unit
Memory Management Unit (MMU)
Matrix Unit
Mapping Operations
MatMul Operations

Mapping Unit

- **Purpose:** Generate a mapping between input point, output point, and weight index based on distance comparisons

Mapping Unit

- **Purpose:** Generate a mapping between input point, output point, and weight index based on distance comparisons
- Traditionally done by querying a hash table

Mapping Unit

- **Purpose:** Generate a mapping between input point, output point, and weight index based on distance comparisons
- Traditionally done by querying a hash table
- PointAcc uses a mergesort-like algorithm

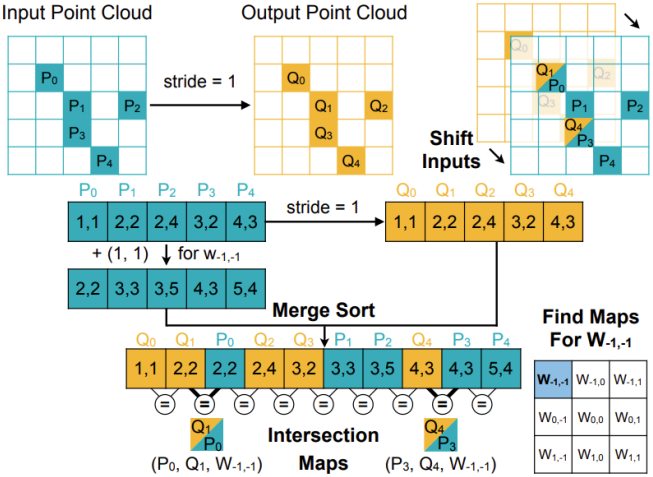
Mapping Unit

- **Purpose:** Generate a mapping between input point, output point, and weight index based on distance comparisons
- Traditionally done by querying a hash table
- PointAcc uses a mergesort-like algorithm
  - Low memory requirements

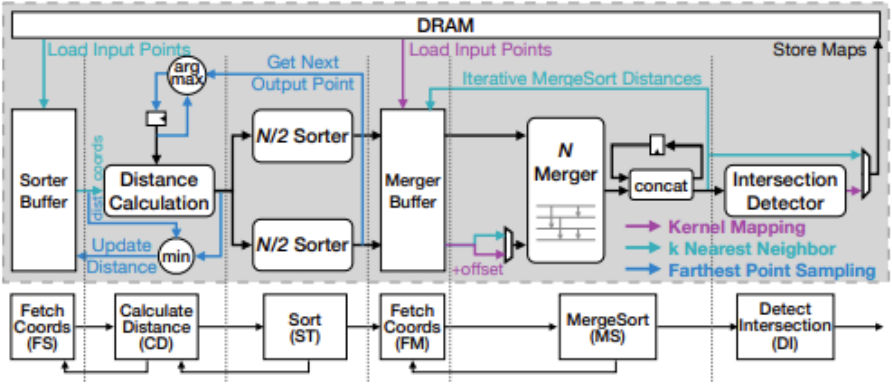
Mapping Unit

- **Purpose:** Generate a mapping between input point, output point, and weight index based on distance comparisons
- Traditionally done by querying a hash table
- PointAcc uses a mergesort-like algorithm
  - Low memory requirements
  - Easily parallelizable

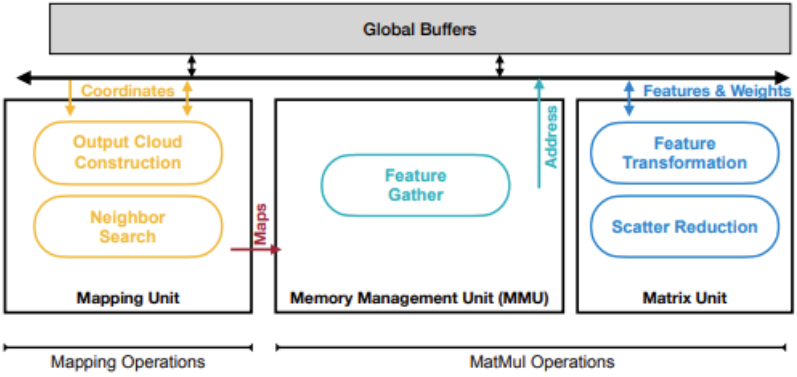
Mergesort


Input Point Cloud
Output Point Cloud
stride = 1
Shift Inputs
P0 P1 P2 P3 P4
1,1 2,2 2,4 3,2 4,3
stride = 1
Q0 Q1 Q2 Q3 Q4
1,1 2,2 2,4 3,2 4,3
+ (1, 1)
for w-1,-1
2,2 3,3 3,5 4,3 5,4
Merge Sort
Q0 Q1 P0 Q2 Q3 P1 P2 Q4 P3 P4
1,1 2,2 2,2 2,4 3,2 3,3 3,5 4,3 4,3 5,4
Intersection Maps
(P0, Q1, W-1,-1)
(P3, Q4, W-1,-1)
Find Maps For W-1,-1
W-1,-1 W-1,0 W-1,1
W0,-1 W0,0 W0,1
W1,-1 W1,0 W1,1

Mapping Unit Architecture



PointAcc


Global Buffers
Coordinates
Features & Weights
Output Cloud Construction
Neighbor Search
Maps
Feature Gather
Address
Feature Transformation
Scatter Reduction
Mapping Unit
Memory Management Unit (MMU)
Matrix Unit
Mapping Operations
MatMul Operations

MMU

- Memory access is slow $\rightarrow$ How to reduce number of DRAM operations?

MMU

- Memory access is slow $\rightarrow$ How to reduce number of DRAM operations?
- Fetch on demand computation flow (reduces miss rate by $3x$)

MMU

- Memory access is slow $\rightarrow$ How to reduce number of DRAM operations?
- Fetch on demand computation flow (reduces miss rate by $3x$)
- Configure input buffers to act as a cache

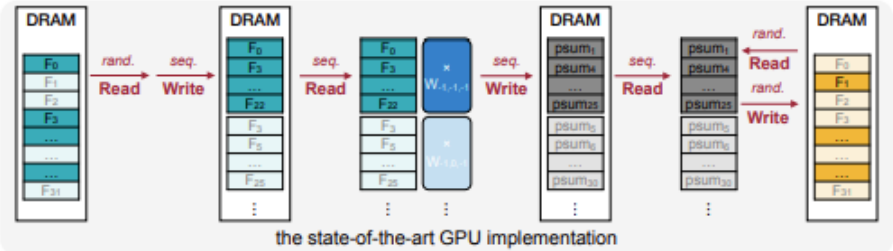
MMU

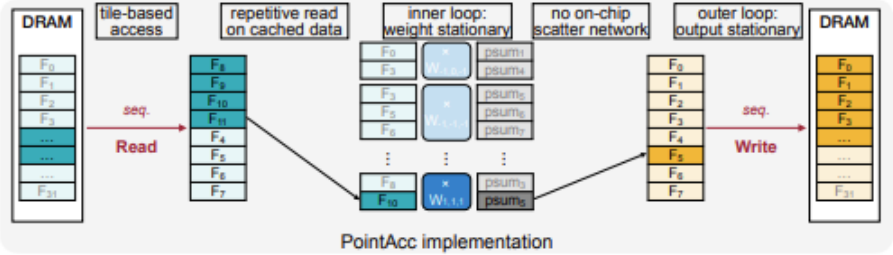
- Memory access is slow $\rightarrow$ How to reduce number of DRAM operations?
- Fetch on demand computation flow (reduces miss rate by $3x$)
- Configure input buffers to act as a cache
- Use 'weight stationary' data flow $\rightarrow$ Process buffered points and neighbors first, then fetch new ones

MMU

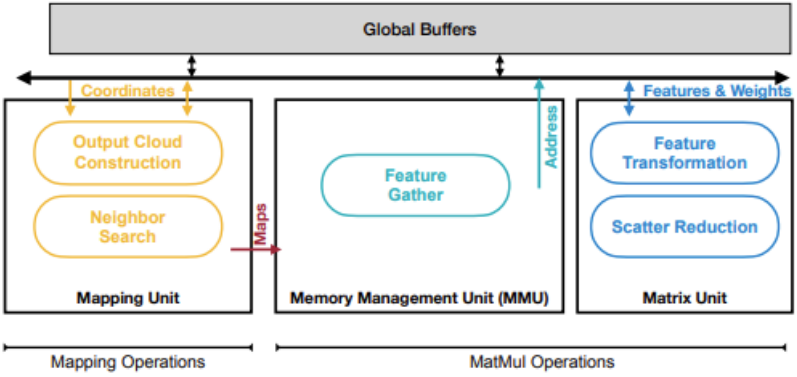
- Memory access is slow $\rightarrow$ How to reduce number of DRAM operations?
- Fetch on demand computation flow (reduces miss rate by $3x$)
- Configure input buffers to act as a cache
- Use 'weight stationary' data flow $\rightarrow$ Process buffered points and neighbors first, then fetch new ones
- Temporally fuse computation of consecutive FCs in point cloud convolution

MMU


DRAM
rand.
Read
seq.
Write
seq.
Read
seq.
Write
seq.
Read
rand.
Read
rand.
Write
the state-of-the-art GPU implementation


DRAM
tile-based access
repetitive read on cached data
inner loop: weight stationary
no on-chip scatter network
outer loop: output stationary
DRAM
seq.
Read
seq.
Write
PointAcc implementation

PointAcc


Global Buffers
Coordinates
Features & Weights
Output Cloud Construction
Neighbor Search
Maps
Feature Gather
Address
Feature Transformation
Scatter Reduction
Mapping Unit
Memory Management Unit (MMU)
Matrix Unit
Mapping Operations
MatMul Operations

Matrix Unit

- Systolic array

Matrix Unit

- Systolic array
- Parallelizes computations in input / output channels

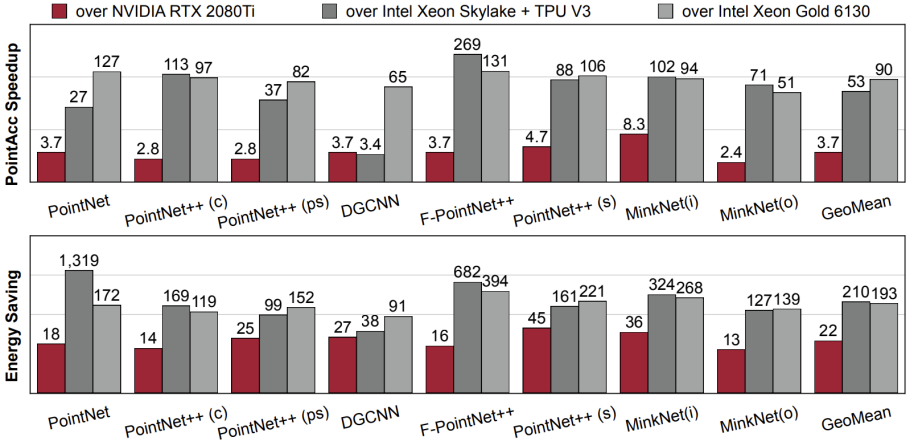
Matrix Unit

- Systolic array
- Parallelizes computations in input / output channels
- Very similar to what we see in TPUs $\rightarrow$ not much new here

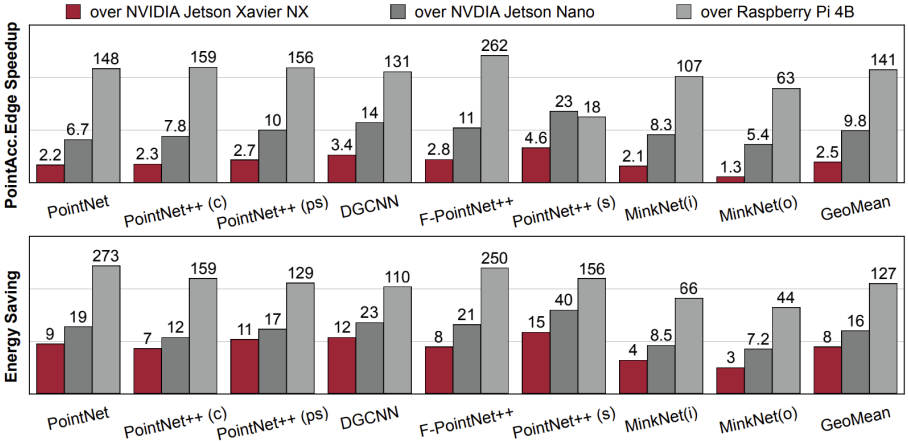
Section 3

Evaluation

Server Products



Edge Devices



Mesorasi

A little friendly competition is always good

- Pre-existing point cloud acceleration from researchers University of Rochester

Mesorasi

A little friendly competition is always good

- Pre-existing point cloud acceleration from researchers University of Rochester
- Not a full-blown hardware accelerator

Mesorasi

A little friendly competition is always good

- Pre-existing point cloud acceleration from researchers University of Rochester
- Not a full-blown hardware accelerator
- Set of minor extensions to CNN accelerators intended to be integrated into SoC systems

Mesorasi

A little friendly competition is always good

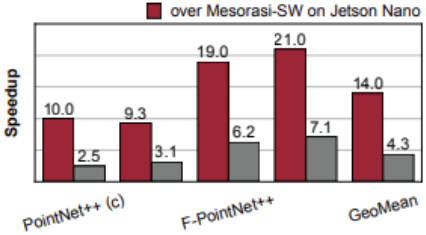
- Pre-existing point cloud acceleration from researchers University of Rochester
- Not a full-blown hardware accelerator
- Set of minor extensions to CNN accelerators intended to be integrated into SoC systems
- Does not support independent weights for different neighbors $\rightarrow$ Limits accuracy

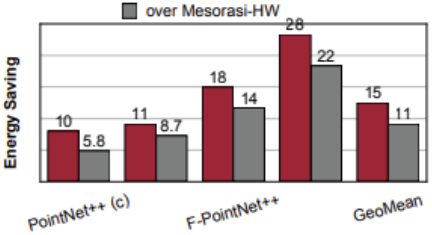
Mesorasi

A little friendly competition is always good

- Pre-existing point cloud acceleration from researchers University of Rochester
- Not a full-blown hardware accelerator
- Set of minor extensions to CNN accelerators intended to be integrated into SoC systems
- Does not support independent weights for different neighbors $\rightarrow$ Limits accuracy
- PointAcc can provide 9.1% higher accuracy (mIoU), 130x lower latency

Mesorasi


over Mesorasi-SW on Jetson Nano
Speedup
10.0
2.5
9.3
3.1
19.0
6.2
21.0
7.1
14.0
4.3
PointNet++ (c)
F-PointNet++
GeoMean


over Mesorasi-HW
Energy Saving
10
5.8
11
8.7
18
14
28
22
15
11
PointNet++ (c)
F-PointNet++
GeoMean

The end.